

Functions in C

What is a Function in C?

A function in C is a named, self-contained block of code that is designed to perform a specific task. It can accept input values called parameters, execute a set of statements, and optionally return a value to the calling program.

Why Do We Need Functions?

Functions are used in C to divide a large program into smaller, manageable blocks of code, where each function performs a specific task.

- **Code Reusability** – Write the code once and use it multiple times.
- **Modularity** – Divide a large program into smaller, independent tasks.
- **Avoid Code Repetition** – The same code does not have to be written again and again.
- **Easy Debugging** – Errors can be identified and fixed in a specific function.
- **Better Readability** – A program becomes easier to understand and maintain.
- **Easy Maintenance** – Changes can be made to one function without affecting the entire program.

Visual Explanation of the Need for Functions

Why We Need to Use Functions in C?

1. Code Reusability

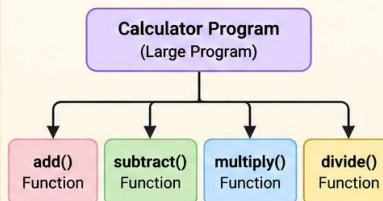
Write the code once and use it multiple times.

```
int add(int a, int b)
{
    return a + b;
}
```

→ add(5, 3)
→ add(10, 20)
→ add(100, 50)

2. Modularity

Divide a large program into smaller, independent tasks.



3. Avoid Code Repetition

The same code does not have to be written again and again.

Without Functions (Repetition)	With Function (No Repetition)
<pre>int result1 = a + b; int result2 = a + b; int result3 = a + b; int result4 = a + b;</pre>	<pre>int add(int a, int b) { return a + b; } int result1 = add(a, b); int result2 = add(a, b); int result3 = add(a, b); int result4 = add(a, b);</pre>

4. Easy Debugging

Errors can be identified and fixed in a specific function.

```
int divide(int a, int b)
{
    return a / b; // Error
}
```

✗ Fix the error in this function only.

5. Better Readability

A program becomes easier to understand.

```
int main()
{
    int a = 10, b = 5;
    int sum = add(a, b);
    int sub = subtract(a, b);
    int mul = multiply(a, b);
    int div = divide(a, b);

    return 0;
}
```

Program is clear and easy to understand with functions.

6. Easy Maintenance

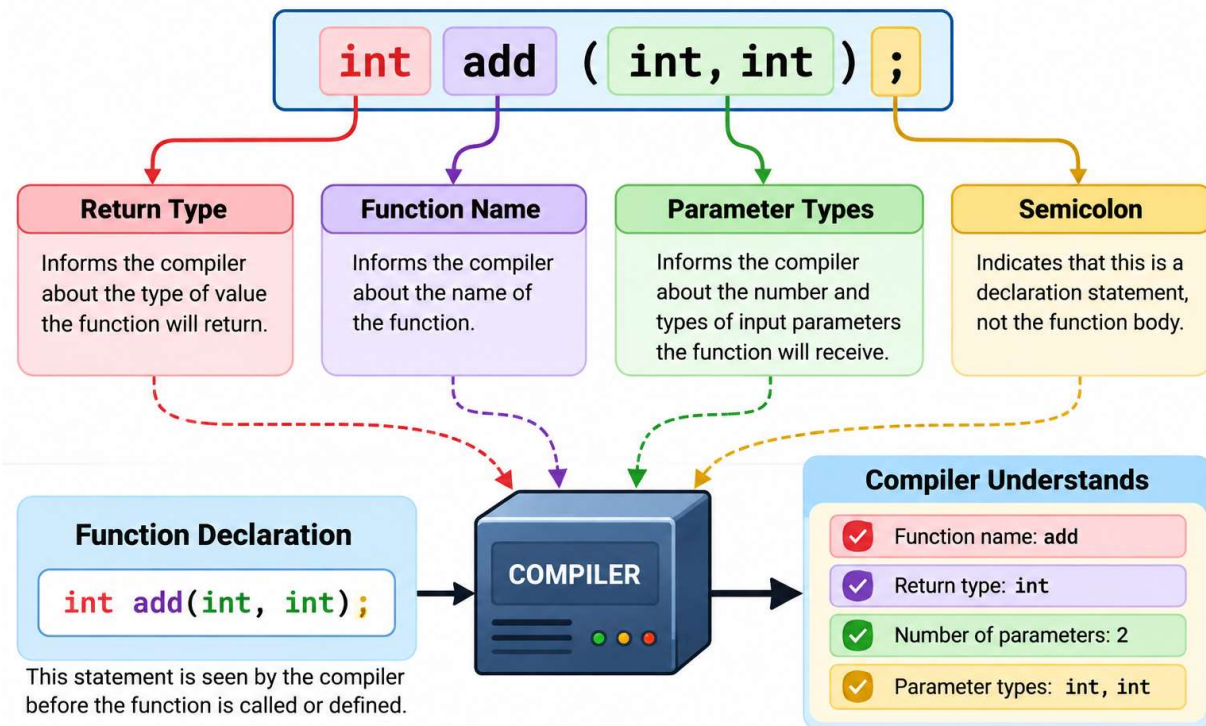
Changes can be made to one function without affecting the entire program.

```
int add(int a, int b)
{
    return a + b;
}
```

→ If we want to change the logic, we update only this function.

Function Declaration in C

A function declaration in C is a statement that informs the compiler about the function's name, return type, and the type and number of parameters before the function is called.



Syntax for the function declaration in C

```
return_type function_name(parameter_list);
```

Where:

- **return_type** – Type of value the function returns.
- **function_name** – Name of the function.
- **parameter_list** – Types of parameters accepted by the function; parameter names are optional.
- **;** – Marks the end of the function declaration.

Examples

With parameters:

```
int add(int, int);
```

With parameter names:

```
int add(int a, int b);
```

Without parameters:

```
int display(void);
```

or

```
int display();
```

Function Definition in C

A function definition in C is the actual implementation of a function that contains the statements required to perform a specific task.

Syntax

```
return_type function_name(parameter_list)
{
    // statements
    // function body
}
```

Example

```
int add(int a, int b)
{
    int sum;
    sum = a + b;
    return sum;
}
```

Function Call in C

A function call is used to tell the program to execute a function.

Syntax

```
function_name(arguments);
```

Example

```
add(10, 20);
```

Function Parameters in C

Parameters are variables written in the function definition that receive values from the function call.

Example

```
int add(int a, int b)
{
    return a + b;
}
```

Here, a and b are parameters.

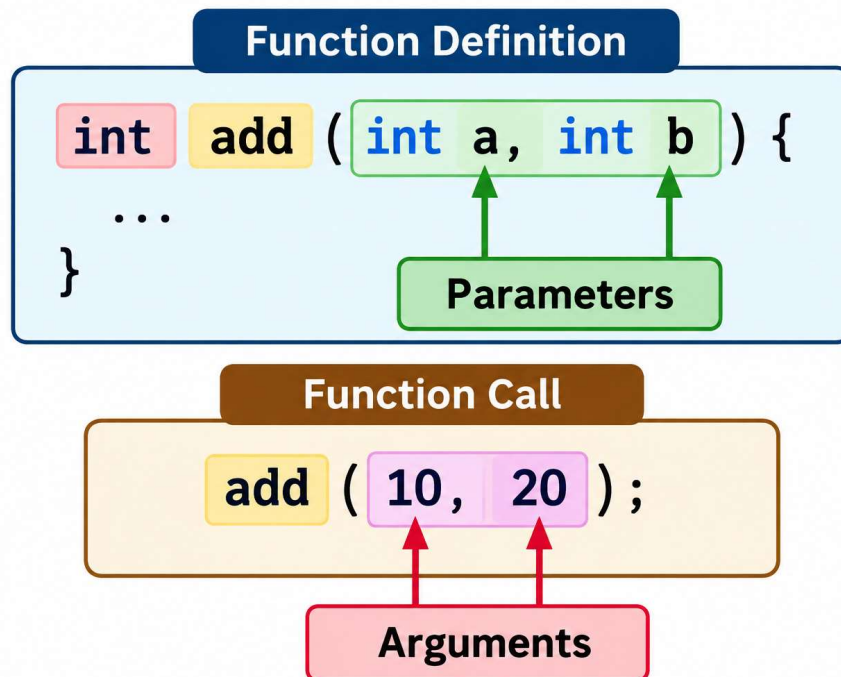
Function Arguments in C

Arguments are the actual values passed to a function when the function is called.

Example

```
add(10, 20);
```

Here, 10 and 20 are arguments.



Return Types in C Functions

A return type specifies the type of value that a function returns to the calling function after completing its task.

Common Return Types

Return Type	Meaning	Example
int	Returns an integer	int add()
float	Returns a decimal value	float average()
double	Returns a larger decimal value	double calculate()
char	Returns a character	char getGrade()
void	Returns no value	void display()

The Four Possible Function Combinations

No.	Function Type	Arguments	Return Value	Example
1	No Arguments, No Return Value	☒ No	☒ No	void greet()
2	With Arguments, No Return Value	☒ Yes	☒ No	void printSum(int a, int b)
3	No Arguments, With Return Value	☒ No	☒ Yes	int getNumber()
4	With Arguments, With Return Value	☒ Yes	☒ Yes	int add(int a, int b)

Visual Summary: The Four Possible Function Combinations

The **Four** Possible Function Combinations in C**1 No Arguments, No Return Value**

Function takes no input and returns no value.

```
void greet()
{
    printf("Hello!");
}
```

Function Call:

```
greet();
```



You press the doorbell (call), it rings (performs task) but gives nothing back.

2 With Arguments, No Return Value

Function takes input but returns no value.

```
void printSum(int a, int b)
{
    printf("Sum = %d", a + b);
}
```

Function Call:

```
printSum(10, 20);
```



You give ingredients (input), the machine makes juice (performs task) but gives nothing back.

3 No Arguments, With Return Value

Function takes no input but returns a value.

```
int getNumber()
{
    return 100;
}
```

Function Call:

```
int n = getNumber();
```



You don't give anything (no input), but it gives you a value (output).

4 With Arguments, With Return Value

Function takes input and returns a value.

```
int add(int a, int b)
{
    return a + b;
}
```

Function Call:

```
int result = add(10, 20);
```



You give numbers (input), the machine calculates and gives a result (output).